AlphaStack: AI-powered Project Generator for Complete Codebases

AlphaStack Team

March 20, 2026

Abstract

This paper presents AlphaStack, a novel approach to autonomous code generation using a multi-agent system with iterative self-healing and comprehensive validation. AlphaStack transforms natural language descriptions into complete, production-ready codebases with Docker configurations and automated testing across diverse programming paradigms. It systematically bridges the gap between language models' raw code-generation capabilities and the reliable creation of full software projects.

1 Introduction

As large language models (LLMs) continue to advance, their capability to generate code snippets has been well-documented. However, building an entire project—including structure, file interconnectivity, build configurations, and tests—remains a substantial challenge. AlphaStack is an AI-driven code generation and validation system that accepts a natural language prompt and fully generates a working software project. Furthermore, it autonomously runs, debugs using an AI Planner Agent, and verifies the generated codebase by running tests inside isolated Docker containers.

2 Methodology

The methodology behind AlphaStack is rooted in a structured pipeline and an agentic loop designed to autonomously catch and resolve software errors.

2.1 7-Phase Generator Pipeline

The core generation process follows a strict 7-phase sequential pipeline:

1. **Software Blueprint**: Analyzes the user prompt and generates a software blueprint defining all files, module roles, and language settings.
2. **File Generation**: Calls the LLM to independently generate actual code for each file based on the blueprint.
3. **Dockerfile Generation**: Generates a production Dockerfile appropriate for the project language and structure.
4. **Dependency Analysis**: Runs static analysis to build an internal dependency graph across all files.

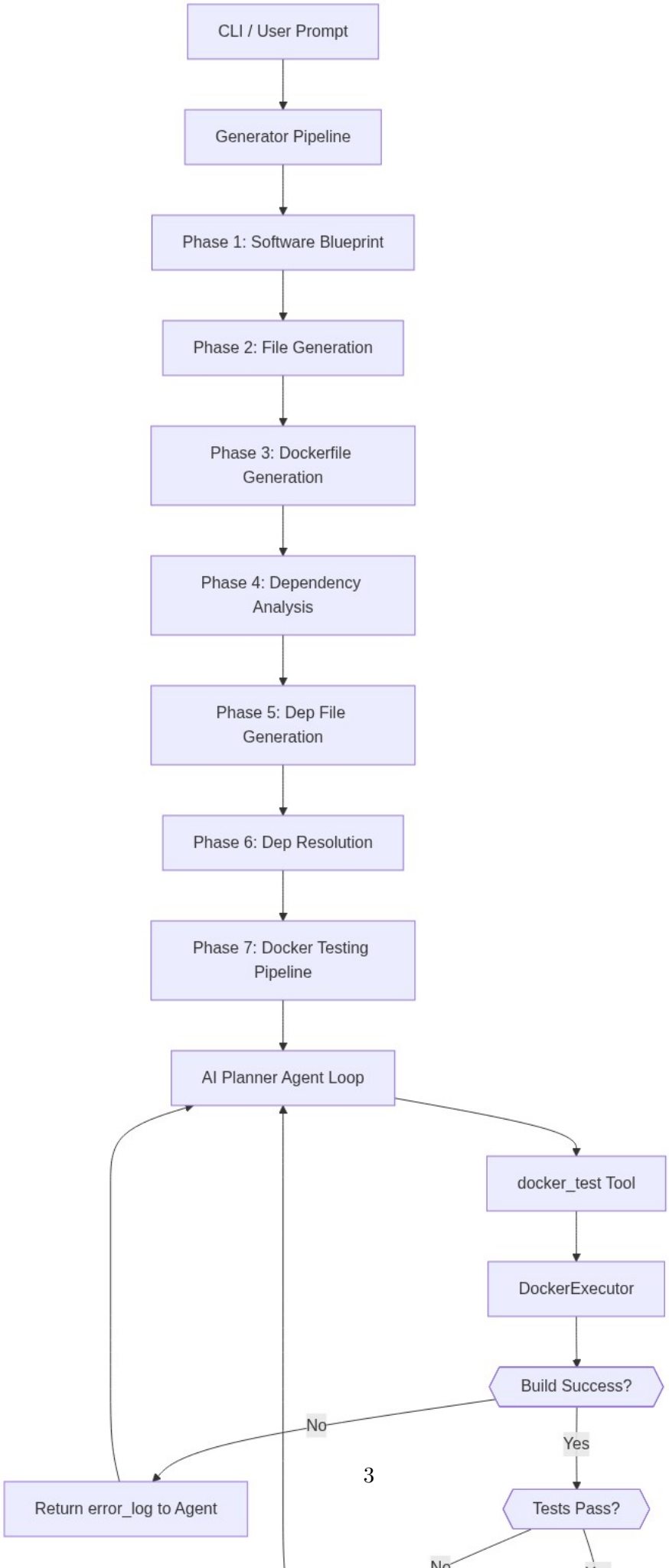
5. **Dep File Generation**: Generates dependency files (e.g., `requirements.txt`, `package.json`, `Cargo.toml`).
6. **Dep Resolution**: Validates the generated dependency files to ensure accuracy.
7. **Docker Testing Pipeline**: Initiates the testing phase and hands control to the AI Planner Agent loop.

2.2 AI Planner Agent Loop

After the initial generation, AlphaStack enters an iterative self-healing loop handled by the Docker Testing Pipeline and the AI Planner Agent. The planner reads the real-time pipeline state, project structure, and tool memory to formulate fixes. It issues tool calls to modify files or interact with the Docker executor (via the `docker_test` tool). The loop continuously builds and tests the project up to a specified iteration limit, exiting successfully only when both the build and tests pass.

3 Architecture Diagram

The architecture of AlphaStack seamlessly integrates the top-level pipeline with the lower-level execution and verification mechanisms. A visual representation of the data flow and system overview is shown in Figure 1.


CLI / User Prompt
Generator Pipeline
Phase 1: Software Blueprint
Phase 2: File Generation
Phase 3: Dockerfile Generation
Phase 4: Dependency Analysis
Phase 5: Dep File Generation
Phase 6: Dep Resolution
Phase 7: Docker Testing Pipeline
AI Planner Agent Loop
docker_test Tool
DockerExecutor
Build Success?
No
Yes
Return error_log to Agent
Tests Pass?

4 Results

AlphaStack has been evaluated on a comprehensive suite of programming challenges. Figure 2 presents tentative results comparing model performance across the HumanEval and MDDP datasets. Models evaluated include GPT-5.2, GLM-5, MiniMax-m2.5, and Claude Sonnet 4.6.

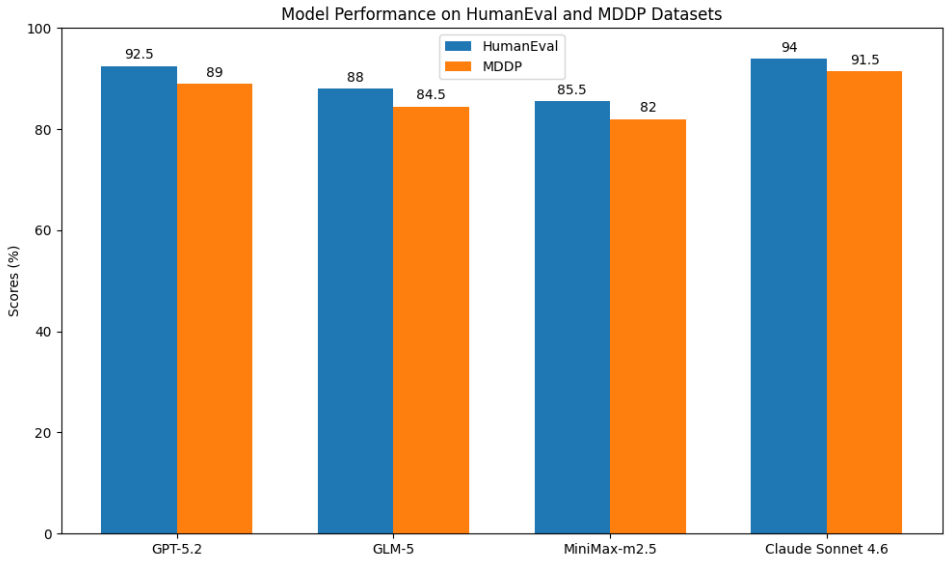


Figure 2: Tentative model performance on HumanEval and MDDP Datasets

5 Conclusion

AlphaStack demonstrates a powerful framework for generating end-to-end software applications from natural language. By coupling a rigorous 7-phase generation pipeline with a dynamic, self-healing AI Planner Agent, the system can reliably construct, test, and repair production-ready codebases. Future work will expand the evaluation framework and incorporate more advanced debugging heuristics.

Supplementary Material

Additional information, including the evaluation datasets (e.g., the 40 challenges across CUDA, Go, Rust, and TypeScript) and extensive logs from the testing pipeline, is available within the open-source repository at `https://github.com/HyperKuvid-Labs/alpha-stack`.